

STAYEASE

Hotel Reservation and Operations Database

Hotel Management Database – Project Report

Item	Details
Team	Team 7
Project	Hotel Management System
System Name	StayEase
Team Size	6 members
RDBMS used	MySQL
Total Marks	40

Team Members

S. No.	Student Name	USN
1	Pavan Kumar	AU25UG-069
2	Praneeth Reddy	AU25UG-044
3	S Mohammed Abrar ul Haq	AU25UG-048
4	Sanghavi S	AU25UG-071
5	Sanika B Raj	AU25UG-052
6	Shafreen	AU25UG-079

1. Problem Statement

A hotel has to manage many types of information at the same time. It needs to keep track of guests, room types, individual rooms, reservations, payments, hotel services, employees, housekeeping activities and guest reviews. If this information is kept together without a proper structure, data can be repeated and it becomes harder to maintain.

For this project, our team developed StayEase, a relational database for managing the main reservation and operational activities of a hotel. The database separates information into related tables and uses primary keys and foreign keys to connect the tables.

The main purpose of StayEase is to:

- store guest profiles and maintain guest stay history;
- define room types with capacity and base pricing;
- maintain individual rooms and their current status;
- create and manage reservations, including check-in, check-out and cancellations;
- record payments associated with reservations;
- record chargeable services used during a stay;
- maintain employee information and housekeeping tasks;
- store guest reviews and ratings; and
- answer useful hotel-management questions through SQL queries.

2. Project Scope and Objectives

- Guest management and stay history.
- Room type and individual room management.
- Reservation, check-in, check-out and cancellation tracking.
- Payment and revenue tracking.
- Restaurant, spa, laundry and other service usage tracking.
- Employee and housekeeping management.
- Guest review and rating analysis.
- Business analysis through SQL queries.
- Demonstration of ER modelling, keys, integrity constraints, functional dependencies, normalization and SQL.

3. Database Design

The StayEase database is designed as a relational database with ten main tables. Each table represents a specific part of the hotel system instead of putting all information into one large table.

3.1 ER Diagram

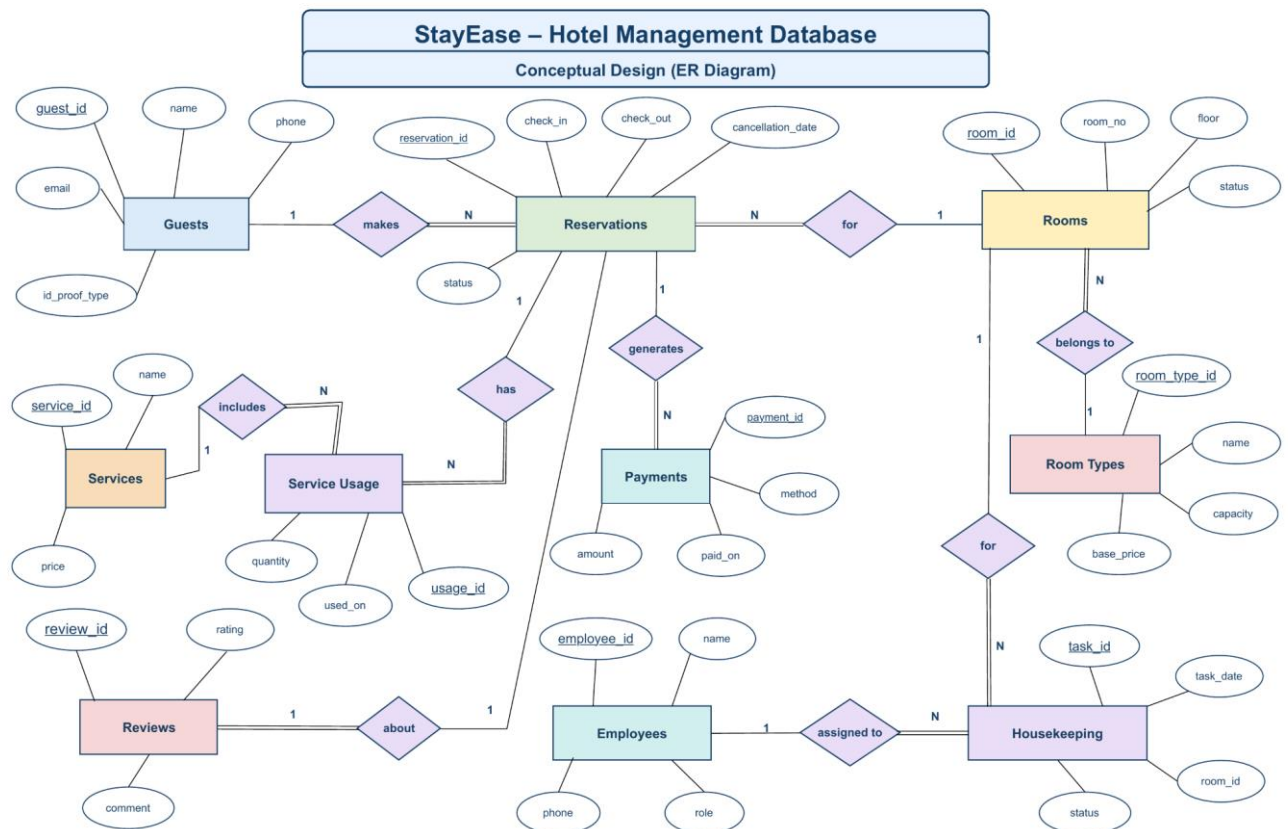


Figure 1. StayEase conceptual ER diagram.

The ER diagram shows the entities, their attributes, primary-key attributes and the relationships between the entities.

3.2 Main Relationships

Relationship	Cardinality	Meaning
Guests → Reservations	1:N	One guest can make multiple reservations.
Reservations → Rooms	N:1	A room can have multiple reservations over time; each reservation refers to one room.
Rooms → Room Types	N:1	Multiple individual rooms can belong to one room type.
Reservations → Service Usage	1:N	One reservation can have multiple service-usage records.
Services → Service Usage	1:N	One service can be used in many reservations.
Reservations → Payments	1:N	A reservation can have payment records.
Reservations → Reviews	1:1	A reservation can have at most one review in the implemented schema.
Employees → Housekeeping	1:N	One employee can be assigned multiple housekeeping tasks.
Rooms → Housekeeping	1:N	A room can have multiple housekeeping tasks over time.

3.3 Main Tables

Table	Important Columns	Purpose
guests	guest_id (PK), name, phone, email, id_proof_type	Stores hotel guest details.
room_types	room_type_id (PK), name, capacity, base_price	Stores room categories and standard pricing.
rooms	room_id (PK), room_no, floor, status, room_type_id (FK)	Stores individual hotel rooms.
reservations	reservation_id (PK), guest_id (FK), room_id (FK), check_in, check_out, status, cancellation_date	Stores room bookings and reservation status.
services	service_id (PK), name, price	Stores chargeable hotel services.
service_usage	usage_id (PK), reservation_id (FK), service_id (FK), quantity, used_on	Resolves reservation-service usage and records quantity/date.
employees	employee_id (PK), name, role, phone	Stores hotel employee information.
housekeeping	task_id (PK), room_id (FK), employee_id (FK), task_date, status	Stores housekeeping tasks and assignments.
payments	payment_id (PK), reservation_id (FK), amount, method, paid_on	Stores payments against reservations.
reviews	review_id (PK), reservation_id (FK), rating, comment	Stores guest feedback and ratings.

4. Database Schema, Keys and Constraints

The final schema is based on the ER design. Primary keys uniquely identify records, while foreign keys connect related records.

- guests(guest_id PK, name, phone, email, id_proof_type)
- room_types(room_type_id PK, name, capacity, base_price)
- rooms(room_id PK, room_no, floor, status, room_type_id FK)
- reservations(reservation_id PK, guest_id FK, room_id FK, check_in, check_out, status, cancellation_date)
- services(service_id PK, name, price)
- service_usage(usage_id PK, reservation_id FK, service_id FK, quantity, used_on)
- employees(employee_id PK, name, role, phone)
- housekeeping(task_id PK, room_id FK, employee_id FK, task_date, status)
- payments(payment_id PK, reservation_id FK, amount, method, paid_on)
- reviews(review_id PK, reservation_id FK, rating, comment)

4.1 Key Design Choices

- Every table has a single-column primary key in the implemented schema.
- rooms.room_no is UNIQUE so two physical rooms cannot have the same room number.
- reviews.reservation_id is UNIQUE, so one reservation can have at most one review.
- Foreign keys maintain referential integrity between related entities.
- Required attributes are marked NOT NULL.
- CHECK constraints enforce valid ranges and business rules.

4.2 Integrity Constraints

```
USE stayease;
```

```
ALTER TABLE room_types
ADD CONSTRAINT chk_room_capacity
CHECK (capacity > 0);
```

```
ALTER TABLE room_types
ADD CONSTRAINT chk_room_base_price
CHECK (base_price >= 0);
```

```
ALTER TABLE services
ADD CONSTRAINT chk_service_price
CHECK (price >= 0);
```

```
ALTER TABLE service_usage
ADD CONSTRAINT chk_service_quantity
CHECK (quantity > 0);
```

```
ALTER TABLE payments
ADD CONSTRAINT chk_payment_amount
CHECK (amount >= 0);
```

```
ALTER TABLE reviews
ADD CONSTRAINT chk_review_rating
CHECK (rating BETWEEN 1 AND 5);
```

```
ALTER TABLE reservations
ADD CONSTRAINT chk_reservation_dates
CHECK (check_out > check_in);
```

These constraints prevent invalid values such as negative prices, non-positive service quantities, ratings outside the 1–5 range, and reservations where check-out is not after check-in.

5. Assumptions

1. Each guest has a unique `guest_id`.
2. Each room has a unique `room_id`.
3. Each room belongs to one room type.
4. A guest can make multiple reservations over time.
5. A room can have multiple reservations over time.
6. Each reservation is associated with one guest and one room.
7. A reservation can have payment records associated with it.
8. A reservation can use multiple services.
9. A service can be used by multiple reservations.
10. Housekeeping tasks are associated with rooms and assigned to employees.
11. Reviews are associated with reservations.
12. The base price stored for a room type represents its standard room price.

6. Design Rationale

6.1 Separation of Room Types and Rooms

The `room_types` and `rooms` tables are kept separate because a room type represents a category, while a room represents an individual physical room. Multiple rooms can share the same room type, avoiding repetition of capacity and base price.

6.2 Separation of Guests and Reservations

Guest information and reservation information are separate because a guest can make multiple reservations over time. This avoids repeating guest details and preserves reservation history.

6.3 Use of Service Usage

Reservations and services can have a many-to-many relationship. The `service_usage` table resolves this relationship and stores quantity and usage date.

6.4 Separate Payment Table

Payment details are different from booking details. The `payments` table stores amount, method and payment date and makes revenue analysis easier.

6.5 Separate Employee and Housekeeping Tables

The `employees` table stores employee details, while `housekeeping` stores individual tasks. This prevents employee information from being repeated for every task.

6.6 Separate Review Table

Reviews store feedback independently from reservation details, allowing rating and comment analysis.

6.7 Addition of Cancellation Date

`cancellation_date` records when a reservation was cancelled, while `status` indicates the reservation state. This supports cancellation analysis.

6.8 Use of Primary and Foreign Keys

Primary keys uniquely identify records and foreign keys establish relationships and maintain referential integrity.

6.9 Normalized Database Structure

The database is divided into related tables rather than one large table. This reduces redundancy and helps prevent insertion, update and deletion anomalies.

7. Functional Dependencies

The functional dependencies used in the design are primarily based on the fact that each primary key uniquely determines the non-key attributes in its relation.

Relation	Primary Key	Functional Dependency
Guests	guest_id	guest_id → name, phone, email, id_proof_type
Room Types	room_type_id	room_type_id → name, capacity, base_price
Rooms	room_id	room_id → room_no, floor, status, room_type_id
Reservations	reservation_id	reservation_id → guest_id, room_id, check_in, check_out, status, cancellation_date
Services	service_id	service_id → name, price
Service Usage	usage_id	usage_id → reservation_id, service_id, quantity, used_on
Payments	payment_id	payment_id → reservation_id, amount, method, paid_on
Reviews	review_id	review_id → reservation_id, rating, comment
Employees	employee_id	employee_id → name, role, phone
Housekeeping	task_id	task_id → room_id, employee_id, task_date, status

8. Normalization

Normalization is used to organize the database and reduce unnecessary data duplication.

8.1 First Normal Form (1NF)

A table is in 1NF when each column contains a single atomic value and there are no repeating groups. In StayEase, each table has separate columns for its attributes and each cell contains one value. For example, the Guests table stores one guest name, phone number, email and ID proof type in separate columns. Therefore, the StayEase relations satisfy 1NF.

8.2 Second Normal Form (2NF)

A table is in 2NF when it is already in 1NF and all non-key attributes depend on the whole primary key. In StayEase, the implemented tables use single-column primary keys such as `guest_id`, `room_id` and `reservation_id`. Since there is only one attribute in each primary key, there cannot be a partial dependency. Therefore, the StayEase relations satisfy 2NF.

8.3 Third Normal Form (3NF)

A table is in 3NF when it is in 2NF and no non-key attribute depends on another non-key attribute.

Room-related transitive dependency:

```
room_id → room_type_id
room_type_id → name, capacity, base_price
```

Rooms:

```
room_id → room_no, floor, status, room_type_id
```

Room_Types:

```
room_type_id → name, capacity, base_price
```

Guest-related dependency:

```
reservation_id → guest_id
guest_id → name, phone, email, id_proof_type
```

Guest details are therefore stored in Guests rather than being repeated in Reservations. The final design is intended to satisfy 3NF and reduce transitive dependency and redundancy.

9. Sample Data Used

Table	Rows
guests	8
room_types	3
rooms	8
services	6
employees	6
reservations	10
service_usage	10
payments	9
reviews	8
housekeeping	8

The sample data was inserted into all ten tables after the schema and constraints were created. It was designed to represent different hotel situations and to provide enough records for meaningful SQL query results.

9.1 Data Coverage

- Guests with more than one reservation, including Aarav and Diya.
- Standard, Deluxe and Suite room types.
- Rooms marked Occupied, Available and Maintenance.
- Reservations with Completed, Cancelled, Checked In and Booked statuses.
- A cancelled reservation with a cancellation date.
- Laundry, Restaurant, Spa, Room Service, Airport Pickup and Breakfast services.
- UPI, Card and Cash payments.
- Guest ratings and comments.
- Completed, In Progress and Pending housekeeping tasks.

9.2 Insertion Order

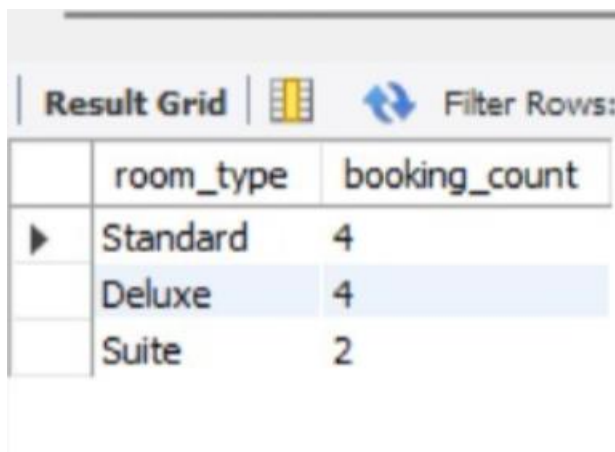
13. guests
14. room_types
15. rooms
16. services
17. employees
18. reservations
19. service_usage
20. payments
21. reviews
22. housekeeping

10. SQL Queries

The following 15 SQL queries answer the required business questions and additional hotel-management questions.

Query 1 – Which room type is booked most often?

```
SELECT
    rt.name AS room_type,
    COUNT(*) AS booking_count
FROM reservations r
JOIN rooms rm ON r.room_id = rm.room_id
JOIN room_types rt ON rm.room_type_id = rt.room_type_id
GROUP BY rt.room_type_id, rt.name
ORDER BY booking_count DESC;
```

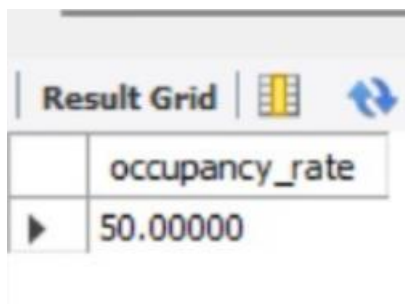


The screenshot shows a 'Result Grid' with two columns: 'room_type' and 'booking_count'. The data is as follows:

	room_type	booking_count
▶	Standard	4
	Deluxe	4
	Suite	2

Query 2 – What is the hotel occupancy rate?

```
SELECT
    COUNT(CASE WHEN status = 'Occupied' THEN 1 END) * 100.0
    / COUNT(*) AS occupancy_rate
FROM rooms;
```



The screenshot shows a 'Result Grid' with one column: 'occupancy_rate'. The data is as follows:

	occupancy_rate
▶	50.00000

Query 3 – What is the average length of stay?

```
SELECT
    AVG(DATEDIFF(check_out, check_in)) AS average_length_of_stay
FROM reservations
WHERE status <> 'Cancelled';
```

Result Grid		Filter Rows:
	average_length_of_stay	
▶	2.6667	

Query 4 – What is the revenue by room type?

```

SELECT
    rt.name AS room_type,
    SUM(p.amount) AS total_revenue
FROM payments p
JOIN reservations r ON p.reservation_id = r.reservation_id
JOIN rooms rm ON r.room_id = rm.room_id
JOIN room_types rt ON rm.room_type_id = rt.room_type_id
GROUP BY rt.room_type_id, rt.name
ORDER BY total_revenue DESC;

```

Result Grid			Filter Rows:
	room_type	total_revenue	
▶	Deluxe	28000.00	
	Standard	26400.00	
	Suite	24000.00	

Query 5 – Which guests have stayed more than once?

```

SELECT
    g.guest_id,
    g.name,
    COUNT(r.reservation_id) AS stay_count
FROM guests g
JOIN reservations r ON g.guest_id = r.guest_id
WHERE r.status <> 'Cancelled'
GROUP BY g.guest_id, g.name
HAVING COUNT(r.reservation_id) > 1;

```

Result Grid			
	guest_id	name	stay_count
▶	1	Aarav	2
	2	Diya	2

Query 6 – Which room types have the highest ratings?

```

SELECT
    rt.name AS room_type,
    AVG(rv.rating) AS average_rating
FROM reviews rv
JOIN reservations r ON rv.reservation_id = r.reservation_id
JOIN rooms rm ON r.room_id = rm.room_id
JOIN room_types rt ON rm.room_type_id = rt.room_type_id
GROUP BY rt.room_type_id, rt.name
ORDER BY average_rating DESC;

```

Result Grid		
	room_type	average_rating
▶	Suite	4.5000
	Standard	4.3333
	Deluxe	4.3333

Query 7 – What is the cancellation rate?

```

SELECT
    COUNT(CASE WHEN status = 'Cancelled' THEN 1 END) * 100.0
    / COUNT(*) AS cancellation_rate
FROM reservations;

```

Result Grid |   Filter Rows:

	cancellation_rate
▶	10.00000

Query 8 – Revenue by payment method

```
SELECT
    method,
    SUM(amount) AS total_revenue
FROM payments
GROUP BY method
ORDER BY total_revenue DESC;
```

Result Grid

	method	total_revenue
▶	UPI	39600.00
	Card	27800.00
	Cash	11000.00

Query 9 – Which services are used most often?

```
SELECT
    s.name AS service,
    SUM(su.quantity) AS total_quantity
FROM service_usage su
JOIN services s ON su.service_id = s.service_id
GROUP BY s.service_id, s.name
ORDER BY total_quantity DESC;
```

Result Grid   Filter Rows:		
	service	total_quantity
▶	Laundry	5
	Breakfast	4
	Restaurant	3
	Spa	2
	Room Service	2
	Airport Pickup	1

Query 10 – Current status of all rooms

```
SELECT
    status,
    COUNT(*) AS room_count
FROM rooms
GROUP BY status
ORDER BY room_count DESC;
```

Result Grid   Filter Rows:		
	status	room_count
▶	Occupied	4
	Available	3
	Maintenance	1

Query 11 – Employees who completed the most housekeeping tasks

```
SELECT
    e.name AS employee_name,
    COUNT(h.task_id) AS completed_tasks
FROM employees e
JOIN housekeeping h ON e.employee_id = h.employee_id
WHERE h.status = 'Completed'
GROUP BY e.employee_id, e.name
ORDER BY completed_tasks DESC;
```

Result Grid   Filter Rows:		
	employee_name	completed_tasks
▶	Ravi	2
	Divya	1
	Arjun	1

Query 12 – Available rooms for each room type

```

SELECT
    rt.name AS room_type,
    COUNT(r.room_id) AS available_rooms
FROM room_types rt
JOIN rooms r ON rt.room_type_id = r.room_type_id
WHERE r.status = 'Available'
GROUP BY rt.room_type_id, rt.name
ORDER BY available_rooms DESC;

```

Result Grid   Filter Rows:		
	room_type	available_rooms
▶	Standard	2
	Deluxe	1

Query 13 – Reservations that used hotel services

```

SELECT
    r.reservation_id,
    g.name AS guest_name,
    COUNT(su.usage_id) AS services_used
FROM reservations r
JOIN guests g ON r.guest_id = g.guest_id
JOIN service_usage su ON r.reservation_id = su.reservation_id
GROUP BY r.reservation_id, g.name
ORDER BY services_used DESC;

```


Result Grid   Filter Rows:			
	reservation_id	guest_name	services_used
▶	1	Aarav	2
	3	Rohan	2
	5	Aarav	1
	2	Diya	1
	8	Diya	1
	4	Meera	1

Query 14 – Total revenue generated by each service

```
SELECT
    s.name AS service,
    SUM(s.price * su.quantity) AS service_revenue
FROM services s
JOIN service_usage su ON s.service_id = su.service_id
GROUP BY s.service_id, s.name
ORDER BY service_revenue DESC;
```

Result Grid   Filter Rows:		
	service	service_revenue
▶	Restaurant	2400.00
	Spa	2400.00
	Breakfast	1600.00
	Laundry	1500.00
	Airport Pickup	1500.00
	Room Service	1000.00

Query 15 – Employees with pending housekeeping tasks

```
SELECT
    e.name AS employee_name,
    COUNT(h.task_id) AS pending_tasks
FROM employees e
JOIN housekeeping h ON e.employee_id = h.employee_id
WHERE h.status = 'Pending'
GROUP BY e.employee_id, e.name
ORDER BY pending_tasks DESC;
```

Result Grid   Filter Rows:		
	employee_name	pending_tasks
▶	Divya	2

11. Summary of Query Results

Q	Business Question	Main Result
1	Most frequently booked room type	Standard – 4; Deluxe – 4; Suite – 2
2	Hotel occupancy rate	50%
3	Average length of stay	2.6667 days
4	Revenue by room type	Deluxe – ₹28,000; Standard – ₹26,400; Suite – ₹24,000
5	Guests with more than one stay	Aarav – 2; Diya – 2
6	Highest room-type ratings	Suite – 4.5; Standard – 4.3333; Deluxe – 4.3333
7	Cancellation rate	10%
8	Revenue by payment method	UPI – ₹39,600; Card – ₹27,800; Cash – ₹11,000
9	Most-used services	Laundry – 5; Breakfast – 4; Restaurant – 3
10	Current room status	Occupied – 4; Available – 3; Maintenance – 1
11	Completed housekeeping tasks	Ravi – 2; Divya – 1; Arjun – 1
12	Available rooms by type	Standard – 2; Deluxe – 1
13	Reservations using services	Reservations with recorded service usage identified
14	Service revenue	Restaurant – ₹2,400; Spa – ₹2,400; Breakfast – ₹1,600
15	Pending housekeeping tasks	Divya – 2

Business Analysis

The SQL analysis helps hotel management understand occupancy, room-type revenue, cancellations, repeat guests, service usage, payment methods and housekeeping performance. These insights can support better decisions on room availability, pricing, service planning and day-to-day hotel operations.

12. Conclusion

StayEase demonstrates how a relational database can be used to organize and manage hotel reservation and operational information. The design separates guests, rooms, room types, reservations, services, service usage, payments, employees, housekeeping and reviews into related tables.

The project applies primary keys, foreign keys, NOT NULL, UNIQUE and CHECK constraints to improve data integrity. Functional dependencies and normalization were used to structure the relations and reduce unnecessary duplication. The sample data covers different hotel operating situations and allows the SQL queries to produce meaningful business results.

The 15 SQL queries demonstrate practical use of JOIN, COUNT, SUM, AVG, GROUP BY, HAVING, conditional aggregation and date functions. Together, the database design and query analysis provide a structured foundation for understanding hotel reservations and operations.